

# Numerikus analízis

## 1. Bevezetés

Ferenc Hartung

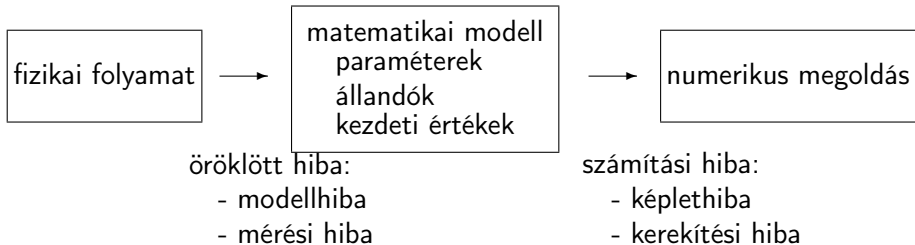
Pannon Egyetem  
Matematika Tanszék



2026

## 1.1. A numerikus analízis feladata, alapfogalmak

Tudományos számítások vázlatos menete:



A **numerikus analízis feladata** matematikai feladatok numerikus eredményének aritmetikai műveletekkel (osztás, szorzás, összeadás, kivonás) való pontos vagy közelítő megoldása.

A **képlethiba** az a hiba, amit akkor követünk el, amikor egy matematikai kifejezés pontos értéke helyett annak közelítő értékét használjuk.

## Példa

Számítsuk ki az  $f(x) = \sin x$  függvény értékét egy  $x$  pontban.

Tekintsük az  $f$  függvény ötödrendű Taylor-polinomját:

$$T_5(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!}.$$

A Taylor-tétel szerint

$$f(x) = T_5(x) + \frac{f^{(6)}(\xi)}{6!}x^6.$$

A képlethiba

$$\frac{f^{(6)}(\xi)}{6!}x^6 = -\frac{\sin \xi}{6!}x^6.$$

A **kerekítési hiba** abból adódik, hogy a számítógépen egy valós számot csak véges sok tizedesjegy pontossággal tudunk tárolni.

A következő fogalom, ami egy numerikus módszerrel kapcsolatban felmerül, a **stabilitás**. Ezt a fogalmat kétféle értelemben is használjuk. Beszélhetünk egy **matematikai modell vagy feladat stabilitásáról**, vagy egy **numerikus módszer stabilitásáról**.

## Példa

Tekintsük a

$$7x + 505y = 940$$

$$6x + 433y = 806$$

lineáris egyenletrendszer. A pontos megoldás  $x = -10$  és  $y = 2$ .

Tekintsük most:

$$7x + 505y = 940$$

$$6.01x + 433y = 806.$$

Ennek megoldása  $x = 2.4691$  és  $y = 1.8272$ . Látható, hogy 0.17%-os változás az egyik együtthatóban a megoldás 124.7% ill. 8.6%-os megváltozását eredményezi.

Azt mondjuk, hogy egy matematikai feladat **korrekt** vagy **stabil**, ha „kis” változás a feladat paramétereiben a megoldás „kis” változását idézi csak elő. Ellenkező esetben **inkorrekt** vagy **instabil feladatról** beszélünk.

Egy numerikus algoritmust a kerekítési hibákra nézve **stabilnak** nevezünk, ha a kerekítési hibák nem befolyásolják jelentősen a számított végeredményt. Ha a kerekítési hibák miatt a számított végeredmény jelentősen eltér a számítandó értéktől, akkor az algoritmust **instabilnak** nevezzük.

## Példa

Tekintsük a következő három, rekurzív definícióval megadott sorozatot:

$$x_n = \frac{1}{3}x_{n-1}, \quad x_0 = 1,$$

$$y_n = 2y_{n-1} - \frac{5}{9}y_{n-2}, \quad y_0 = 1, \quad y_1 = \frac{1}{3},$$

$$z_n = \frac{13}{3}z_{n-1} - \frac{4}{3}z_{n-2}, \quad z_0 = 1, \quad z_1 = \frac{1}{3}.$$

Könnyen látható, hogy  $x_n = y_n = z_n = \frac{1}{3^n}$ , azaz a három sorozat algebrailag ekvivalens.

*Egyszeres pontosságú* aritmetikával számolva kapjuk:

## Példa folyt.

| $n$ | $x_n$    | $y_n$    | $ y_n - 1/3^n $ | $z_n$      | $ z_n - 1/3^n $ |
|-----|----------|----------|-----------------|------------|-----------------|
| 2   | 0.111111 | 0.111111 | 2.2352e-08      | 0.111111   | 4.4703e-08      |
| 3   | 0.037037 | 0.037037 | 4.0978e-08      | 0.037037   | 1.8254e-07      |
| 4   | 0.012346 | 0.012346 | 6.9849e-08      | 0.012346   | 7.3109e-07      |
| 5   | 0.004115 | 0.004115 | 1.1688e-07      | 0.004118   | 2.9248e-06      |
| 6   | 0.001372 | 0.001372 | 1.9465e-07      | 0.001383   | 1.1699e-05      |
| 7   | 0.000457 | 0.000458 | 3.2442e-07      | 0.000504   | 4.6795e-05      |
| 8   | 0.000152 | 0.000153 | 5.4071e-07      | 0.000340   | 1.8718e-04      |
| 9   | 0.000051 | 0.000052 | 9.0117e-07      | 0.000800   | 7.4872e-04      |
| 10  | 0.000017 | 0.000018 | 1.5019e-06      | 0.003012   | 2.9949e-03      |
| 11  | 0.000006 | 0.000008 | 2.5032e-06      | 0.011985   | 1.1980e-02      |
| 12  | 0.000002 | 0.000006 | 4.1721e-06      | 0.047920   | 4.7918e-02      |
| 13  | 0.000001 | 0.000008 | 6.9535e-06      | 0.191674   | 1.9167e-01      |
| 14  | 0.000000 | 0.000012 | 1.1589e-05      | 0.766693   | 7.6669e-01      |
| 15  | 0.000000 | 0.000019 | 1.9315e-05      | 3.066773   | 3.0668e+00      |
| 16  | 0.000000 | 0.000032 | 3.2192e-05      | 12.267091  | 1.2267e+01      |
| 17  | 0.000000 | 0.000054 | 5.3653e-05      | 49.068363  | 4.9068e+01      |
| 18  | 0.000000 | 0.000089 | 8.9422e-05      | 196.273453 | 1.9627e+02      |

Példa folyt.

*Dupla pontosságú* aritmetikával számolva a 18. tag hibája:

$$|y_{18} - 1/3^{18}| = -2.5104\text{e-}13 \quad \text{és} \quad |z_{18} - 1/3^{18}| = 2.3804\text{e-}07.$$

A következő fogalom, amit egy (véges sok lépésből álló) numerikus módszernél vizsgálni szoktunk, az algoritmus **műveletigénye** vagy **műveletszáma**.

## Példa

Számítsuk ki a  $p(x) = 5x^4 - 8x^3 + 2x^2 + 4x - 10$  negyedfokú polinom értékét egy megadott  $x$  pontban! A képletben 4 összeadás/kivonás, 4 szorzás és 3 hatványozás szerepel. A hatványozások tulajdonképpen  $3+2+1=6$  szorzást jelentenek, azaz összesen 10 szorzásra van szükség a képlet alkalmazásához. Megtehetjük viszont, hogy átalakítjuk  $p$  képletét:

$$p(x) = 5x^4 - 8x^3 + 2x^2 + 4x - 10 = (((5x - 8)x + 2)x + 4)x - 10.$$

A  $p$  polinomnak ezt az alakját használva már csak 4 összeadás ill. kivonás valamint 4 szorzás kell a képlet kiértékeléséhez.

Az előző példában bemutatott eljárást megismételhetjük általános  $n$ -edfokú polinomokra:

$$a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0 = (((\dots ((a_n x + a_{n-1})x + a_{n-2})x + \dots)x + a_1)x + a_0$$

Ebben a képletben összesen csak  $n$  összeadás/kivonás és  $n$  szorzás szerepel. Ezt a polinomok kiértékelésére vonatkozó módszert

**Horner-eljárásnak** nevezzük.

### Algoritmus: Horner-módszer

---

INPUT:  $n$  - a polinom fokszáma  
 $a_n, a_{n-1}, \dots, a_0$  - a polinom együtthatói  
 $x$  - ahol a polinomot kiértékeljük  
 OUTPUT:  $p$  - a polinom értéke az  $x$  pontban

```

 $p \leftarrow a_n$ 
for  $i = n - 1, \dots, 0$  do
     $p \leftarrow a_i + px$ 
end do
output( $p$ )
  
```

---

Egy algoritmusra jellemző tulajdonság még az **adattárolási igénye**. Egy  $10 \times 10$ -es lineáris egyenletrendszer megoldására használt algoritmus esetében az adatok tárolása nem jelenthet problémát, de ugyanez  $10000 \times 10000$ -es rendszerre már gond lehet. Ilyen mennyiségű adat kezelésekor előnyben részesítünk olyan algoritmusokat, amelyeknek minél kisebb az adattárolási igénye.

## 1.2. Valós számok tárolása

A  $b$  alapú számrendszerben felírt

$$x = (x_{m-1}x_{m-2} \cdots x_0.x_{-1}x_{-2} \cdots)_b, \quad x_i \in \{0, 1, \dots, b-1\},$$

valós szám értéke

$$x = x_{m-1}b^{m-1} + x_{m-2}b^{m-2} + \cdots + x_1b + x_0 + \frac{x_{-1}}{b} + \frac{x_{-2}}{b^2} + \cdots = \sum_{i=-\infty}^{m-1} x_i b^i.$$

Tekintsük a 126.42 valós számot. Ennek normál alakján vagy az  $1.2642 \cdot 10^2$  vagy pedig a  $0.12642 \cdot 10^3$  alakot szokás érteni. Mi az első alakot fogjuk használni.

Egy  $x \neq 0$  valós szám  $b$  alapra vonatkozó **normál alakja**

$$x = \pm m \cdot b^k, \quad \text{ahol } 1 \leq m < b.$$

$m$ -et a szám **mantisszájának**,  $k$ -t pedig **kitevőjének** nevezzük.

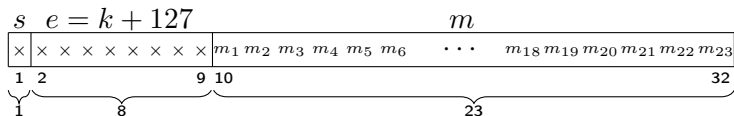
Valós számok, más szóval **lebegőpontos számok** tárolásához a számot felírjuk (valamely  $b$  alapot használva) normál alakban, és az előjeles mantisszát, valamint a kitevőt tároljuk. Különböző számítógépek eltérő alapot és bithosszúságot használnak egy valós szám tárolására. Mi most egy IEEE szabványt (IEEE Binary Floating Point Arithmetic Standard, 754-1985) ismertetünk valós számok 32 biten (ún. **egyszeres pontosságú**), ill. 64 biten történő (ún. **dupla pontosságú**) tárolására bináris alapot használva.

## Valós számok 32 biten történő tárolása:

Vegyük az  $x \neq 0$  szám bináris normál alakját:

$$x = (-1)^s m \cdot 2^k, \quad \text{ahol } s \in \{0, 1\} \text{ és } m = (1.m_1 m_2 m_3 \dots)_2.$$

Az  $s$  értékét az 1. biten tároljuk. A  $k$  kitevő helyett annak eltolt értékét, az  $e = k + 127$  nemnegatív számot a 2–9. biteken tároljuk, a mantissza törtrészének első 23 bitre kerekített értékét pedig a 10–32. biten tároljuk. A nemnulla szám mantisszájának egész része bináris normál alakban mindig 1-gyel egyenlő, ezt az 1-et nem tároljuk!



A fent említett IEEE szabvány külön definiálja a 0 tárolását, és bevezet két speciális szimbólumot is, az  $\text{Inf}$  (infinity, azaz végtelen) és  $\text{NaN}$  (not-a-number, azaz nem szám) szimbólumokat:

| tárolandó szám | $s$ | $e$ (2.–9. bitek) | mantissza bitek (10.–32. bitek) |
|----------------|-----|-------------------|---------------------------------|
| +0             | 0   | 00000000          | minden mantissza bit=0          |
| −0             | 1   | 00000000          | minden mantissza bit=0          |
| +Inf           | 0   | 11111111          | van 0 mantissza bit             |
| −Inf           | 1   | 11111111          | van 0 mantissza bit             |
| +NaN           | 0   | 11111111          | minden mantissza bit=1          |
| −NaN           | 1   | 11111111          | minden mantissza bit=1          |

Az  $\text{Inf}$  szimbólumot a programok használhatják olyan matematikai művelet eredményének tárolására, amelynek értéke végtelen, a  $\text{NaN}$  szimbólumot pedig olyan művelet „eredményének” tárolására, amely nem definiált (pl. nullával való osztás eredménye vagy negatív szám négyzetgyöke valós számok körében).

A szabvány definíciójából következik, hogy az  $e = (1111111)_2 = 255$ , azaz a  $k = 128$  kitevő az Inf és NaN speciális szimbólumoknak van fenntartva. A véges valós számok esetén

$$0 \leq e = k + 127 \leq 254,$$

így  $k$  lehetséges értékei

$$-127 \leq k \leq 127.$$

A legkisebb pozitív valós szám tehát a

$$k = -127 \quad \text{és} \quad m = (1.00 \dots 01)_2$$

értékekhez tartozik. Ennek értéke

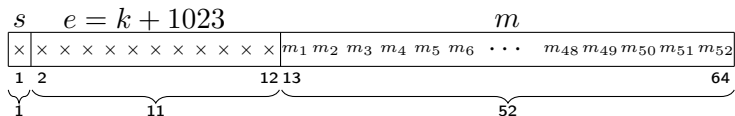
$$x_{\min} = (1.00 \dots 01)_2 \cdot 2^{-127} = (1 + 2^{-23})2^{-127} \approx 10^{-38}.$$

A legnagyobb (véges) valós szám pedig

$$x_{\max} = (1.11 \dots 1)_2 \cdot 2^{127} = (2 - 2^{-23})2^{127} \approx 10^{38}.$$

64 biten törtéző tárolás hasonló:

Az  $e = k + 1023$  eltolt kitevőt a 2.–12. biten, a mantissza törtrészét pedig a 13.–64. biten tároljuk.

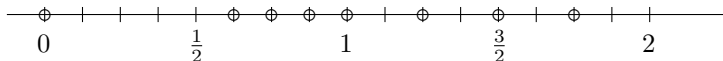


Ekkor a tárolható pozitív számok tartománya körülbelül  $10^{-308} - 10^{308}$  lesz.

## Példa

Tegyük fel, hogy 4 biten szeretnénk valós számokat tárolni, bináris normál alak segítségével, úgy, hogy az 1. biten a szám előjelét, a 2. biten a bináris normál alak eltolt kitevőjét,  $e = k + 1$ -et, a 3–4. biten pedig a mantissza tört részének első két bitjét tároljuk. (Az Inf és NaN szimbólumokat nem definiáljuk most.)

| $s$ | $e$ | $m$ | $x$                                                                                |
|-----|-----|-----|------------------------------------------------------------------------------------|
| 0   | 0   | 00  | 0                                                                                  |
| 0   | 0   | 01  | $(1.01)_2 \cdot 2^{-1} = (1 + \frac{1}{4})\frac{1}{2} = \frac{5}{8}$               |
| 0   | 0   | 10  | $(1.10)_2 \cdot 2^{-1} = (1 + \frac{1}{2})\frac{1}{2} = \frac{3}{4} = \frac{6}{8}$ |
| 0   | 0   | 11  | $(1.11)_2 \cdot 2^{-1} = (1 + \frac{1}{2} + \frac{1}{4})\frac{1}{2} = \frac{7}{8}$ |
| 0   | 1   | 00  | $(1.00)_2 \cdot 2^0 = 1 = \frac{8}{8}$                                             |
| 0   | 1   | 01  | $(1.01)_2 \cdot 2^0 = 1 + \frac{1}{4} = \frac{10}{8}$                              |
| 0   | 1   | 10  | $(1.10)_2 \cdot 2^0 = 1 + \frac{1}{2} = \frac{12}{8}$                              |
| 0   | 1   | 11  | $(1.11)_2 \cdot 2^0 = 1 + \frac{1}{2} + \frac{1}{4} = \frac{7}{4} = \frac{14}{8}$  |



Azokat a számokat, amelyeket pontosan, azaz tárolási hiba nélkül tudunk tárolni, **gépi számoknak** nevezzük. Azt a gépi számot, amelyet egy adott  $x$  valós szám helyett tárolunk a számítógépen, **fl( $x$ )**-szel jelöljük.

$$\text{fl}(x) = \begin{cases} 0, & |x| < x_{\min}, \\ \text{Inf}, & x > x_{\max}, \\ -\text{Inf}, & x < -x_{\max}. \end{cases}$$

Az első esetben **alácsordulásról**, a másodikban **túlcsordulásról** beszélünk. A többi esetben két alapvető megközelítés lehetséges. Az egyik esetben vesszük az  $x$  szám bináris normál alakját, és annak mantisszájából annyi bitet tárolunk, amennyit az adott tárolási rendszerben tudunk, a többit elhagyjuk. Ezt a stratégiát **levágásnak** nevezzük.

A **kerekítés** esetében  $\text{fl}(x)$ -et definiáljuk úgy, hogy legyen az  $x$ -hez legközelebbi gépi szám. A kerekítési szabályt az egyszeres pontosságú tárolásra fogalmazzuk meg. Legyen az  $x > 0$  szám bináris normál alakja

$$x = m2^k, \quad \text{ahol} \quad m = (1.m_1m_2 \dots m_{23}m_{24} \dots)_2.$$

Legyen

$$x' = (1.m_1m_2 \dots m_{23})_2 2^k \quad \text{és} \quad x'' = ((1.m_1m_2 \dots m_{23})_2 + 2^{-23}) 2^k.$$

Ekkor  $x'$  és  $x''$  egymás utáni gépi számok, és

$$x' \leq x \leq x'', \quad \text{valamint} \quad x'' - x' = 2^{k-23}.$$

A szabvány szerint legyen

$$\text{fl}(x) = \begin{cases} x', & \text{ha } |x - x'| < \frac{1}{2}|x'' - x'|, \\ x'', & \text{ha } |x - x''| < \frac{1}{2}|x'' - x'|, \\ x', & \text{ha } |x - x'| = \frac{1}{2}|x'' - x'| \text{ és } m_{23} = 0, \\ x'', & \text{ha } |x - x'| = \frac{1}{2}|x'' - x'| \text{ és } m_{23} = 1. \end{cases}$$

Kerekítést használva tehát az elkövetett hiba

$$|x - \text{fl}(x)| \leq \frac{1}{2}|x'' - x'| = \frac{1}{2}2^{-23}2^k.$$

Vizsgáljuk most a kerekítési hibát a pontos értékhez viszonyítva:

$$\frac{|x - \text{fl}(x)|}{|x|} \leq \frac{|x - \text{fl}(x)|}{(1.m_1m_2\dots)_2 \cdot 2^k} \leq \frac{1}{2}2^{-23}.$$

Könnyen látható, hogy az 1 gépi szám után következő első gépi szám  $1 + 2^{-23}$  az előbb vizsgált 32 bites számábrázolási rendszerben. Ezt általánosítva jelölje  $\varepsilon_m$  az adott számábrázolási rendszerben az első 1-nél nagyobb gépi szám és 1 különbségét. Ezt a számot **gépi epszilonnak** nevezzük. Eszerint  $\varepsilon_m$  a legkisebb olyan 2 hatvány, amelyre a számítógépen az  $1 + \varepsilon_m > 1$  egyenlőtlenség ellenőrizhető.

## Theorem

*Legyen  $0 < \text{fl}(x) < \text{Inf}$ , és tegyük fel, hogy a valós számokat kerekítve tároljuk. Ekkor*

$$\frac{|x - \text{fl}(x)|}{|x|} \leq \frac{1}{2}\varepsilon_m.$$

Legyen  $x$  egy valós szám, és tekintsük az  $\tilde{x}$  valós számot  $x$  közelítésének.  
Ekkor a **közelítés hibáján** az

$$|x - \tilde{x}|$$

számot értjük.

### Példa

Legyen  $x = 10000$  és  $\tilde{x} = 10000.1$ . Ekkor a hiba  $|x - \tilde{x}| = 0.1$ .

Legyen  $y = 1$  és  $\tilde{y} = 1.1$ . Ekkor a hiba  $|y - \tilde{y}| = 0.1$ .

A **közelítés relatív hibáján** az

$$\frac{|x - \tilde{x}|}{|x|} \quad (x \neq 0)$$

számot értjük.

### Példa

Legyen  $x = 10000$  és  $\tilde{x} = 10000.1$ . Ekkor a relatív hiba  $|x - \tilde{x}|/x = 10^{-5}$ .

Legyen  $y = 1$  és  $\tilde{y} = 1.1$ . A relatív hiba  $|y - \tilde{y}|/y = 10^{-1}$ .

Későbbi példáinkban gyakran fogunk hivatkozni az ún. **négyjegyű aritmetikára**. Ez azt jelenti, hogy minden egyes részletszámolás eredményét az első nem nulla számjegytől számított 4 jegyre, azaz az első 4 **értékes számjegyre** kerekítjük, és ezt használjuk tovább a számolás során.

## Példa

Négyjegyű aritmetikát használva

$$1.043 + 32.25 = 33.29,$$

(a pontos érték 33.293) és hasonlóan

$$1.043 \cdot 32.25 = 33.64$$

(a pontos érték 33.63675 kerekítése után). Viszont

$$1.043 + 20340 = 20340$$

lesz, mivel négy értékes számjegyre kell kerekítenünk a 20341.043 pontos értéket.

## 1.3. Hibaanalízis

Legyen  $x$  és  $y$  pozitív, és tekintsük az  $\tilde{x}$  és  $\tilde{y}$  számokat  $x$  és  $y$  közelítésének. Legyen

$$|x - \tilde{x}| \leq \Delta_x \quad \text{és} \quad |y - \tilde{y}| \leq \Delta_y$$

a közelítések hibakorlátja. A megfelelő relatív hibakorlátokat

$$\delta_x := \frac{\Delta_x}{x} \quad \text{és} \quad \delta_y := \frac{\Delta_y}{y},$$

jelölik.

A következő kérdéssel foglalkozunk: az  $x$  és  $y$  számokon egy aritmetikai műveletet (összeadás, kivonás, szorzás, osztás) kell elvégeznünk, és ahelyett az  $\tilde{x}$  és  $\tilde{y}$  számokon végezzük el a műveletet, és annak (pontos) eredményével közelítjük az eredeti művelet eredményét. Adjuk meg, legfeljebb mekkora lehet a közelítés hibája ill. relatív hibája.

Tekintsük az összeadást. Keresünk  $\Delta_{x+y}$  és  $\delta_{x+y}$  korlátokat, amelyekre

$$|x + y - (\tilde{x} + \tilde{y})| \leq \Delta_{x+y} \quad \text{és} \quad \frac{|x + y - (\tilde{x} + \tilde{y})|}{x + y} \leq \delta_{x+y}.$$

## Theorem

A

$$\Delta_{x+y} := \Delta_x + \Delta_y \quad \text{és} \quad \delta_{x+y} := \max\{\delta_x, \delta_y\}$$

*számok az összeadás hiba- ill. relatív hibakorlátai.*

## Bizonyítás

A háromszög-egyenlőtlenséget és  $\Delta_x$  és  $\Delta_y$  definícióját alkalmazva

$$|x + y - (\tilde{x} + \tilde{y})| \leq |x - \tilde{x}| + |y - \tilde{y}| \leq \Delta_x + \Delta_y.$$

Ebből kapjuk hogy  $\Delta_x + \Delta_y$  egy hibakorlátja lesz az összeadásnak.

Az előbbi összefüggést felhasználva

$$\begin{aligned} \frac{|x + y - (\tilde{x} + \tilde{y})|}{x + y} &\leq \frac{\Delta_x + \Delta_y}{x + y} = \frac{\Delta_x}{x + y} + \frac{\Delta_y}{x + y} \\ &= \frac{x}{x + y} \frac{\Delta_x}{x} + \frac{y}{x + y} \frac{\Delta_y}{y} = \frac{x}{x + y} \delta_x + \frac{y}{x + y} \delta_y \\ &\leq \max\{\delta_x, \delta_y\}. \end{aligned}$$

Tehát  $\max\{\delta_x, \delta_y\}$  egy relatív hibakorlátja az összeadásnak.

A tétel a legrosszabb esetre vonatkozik. A gyakorlatban a hibák kiegyenlíthetik egymást.

### Példa

Legyen  $x = 1$ ,  $y = 2$ ,  $\tilde{x} = 1.1$  és  $\tilde{y} = 1.8$ . Ekkor  $x + y = 3$  és  $\tilde{x} + \tilde{y} = 2.9$ , így az összeadás hibája

$$|x + y - (\tilde{x} + \tilde{y})| = 0.1,$$

de

$$\Delta_x + \Delta_y = 0.1 + 0.2 = 0.3.$$

## Theorem

Legyen  $x > y > 0$ . A

$$\Delta_{x-y} := \Delta_x + \Delta_y \quad \text{és} \quad \delta_{x-y} := \frac{x}{x-y} \delta_x + \frac{y}{x-y} \delta_y$$

számok a kivonás hiba- ill. relatív hibakorlátai.

## Bizonyítás

Az

$$|x - y - (\tilde{x} - \tilde{y})| \leq |x - \tilde{x}| + |y - \tilde{y}| \leq \Delta_x + \Delta_y$$

egyenlőtlenségekből következik az első állítás. Tekintsük az

$$\begin{aligned} \frac{|x - y - (\tilde{x} - \tilde{y})|}{x - y} &\leq \frac{\Delta_x + \Delta_y}{x - y} \\ &= \frac{x}{x - y} \frac{\Delta_x}{x} + \frac{y}{x - y} \frac{\Delta_y}{y} \\ &= \frac{x}{x - y} \delta_x + \frac{y}{x - y} \delta_y, \end{aligned}$$

becsléseket, amiből a második állítást kapjuk.

Látható, hogy ha egymáshoz közeli számokat vonunk ki egymásból, akkor a relatív hiba megsokszorozódhat, azaz a pontos számjegyek száma jelentősen csökkenhet. Ezt a jelenséget hívjuk értékes számjegyek végzetes elvesztésének.

### Példa

Legyen  $x = 12.47531$ ,  $\tilde{x} = 12.47534$ ,  $y = 12.47326$ ,  $\tilde{y} = 12.47325$ , akkor

$$\delta_x = 2.4 \cdot 10^{-6} \quad \text{és} \quad \delta_y = 8 \cdot 10^{-7}.$$

Viszont

$$x - y = 0.00205, \quad \tilde{x} - \tilde{y} = 0.00209,$$

így

$$\delta_{x-y} = 0.0195.$$

## Theorem

Legyen  $x, y > 0$ . A

$\Delta_{x \cdot y} := x\Delta_y + y\Delta_x + \Delta_x\Delta_y$ , és  $\delta_{x \cdot y} := \delta_x + \delta_y + \delta_x\delta_y$   
 számok a szorzás hiba- ill. relatív hibakorlátai.

## Bizonyítás

A háromszög-egyenlőtlenség szerint

$$\begin{aligned}
 |xy - \tilde{x}\tilde{y}| &= |xy - x\tilde{y} + x\tilde{y} - \tilde{x}\tilde{y}| \\
 &\leq x|y - \tilde{y}| + |\tilde{y}||x - \tilde{x}| \\
 &\leq x\Delta_y + |\tilde{y}|\Delta_x \\
 &= x\Delta_y + |y + \tilde{y} - y|\Delta_x \\
 &\leq x\Delta_y + y\Delta_x + \Delta_x\Delta_y.
 \end{aligned}$$

Az első állítás szerint a szorzat relatív hibája

$$\frac{|xy - \tilde{x}\tilde{y}|}{xy} \leq \frac{x\Delta_y + y\Delta_x + \Delta_x\Delta_y}{xy} = \delta_x + \delta_y + \delta_x\delta_y.$$

Mivel  $\Delta_x$  és  $\Delta_y$  általában sokkal kisebb mint  $x$  és  $y$ , és így  $\Delta_x \Delta_y$  elhanyagolható  $x\Delta_y$  és  $y\Delta_x$ -hez képest, ezért

$$x\Delta_y + y\Delta_x$$

egy jó becslés a szorzat hibájára. Hasonlóan,

$$\delta_x + \delta_y$$

jó közelítése a szorzat relatív hibakorlátjának.

## Theorem

Tegyük fel, hogy  $x, y > 0$  és  $\delta_y < 1$ . Ekkor a

$$\Delta_{x/y} := \frac{x\Delta_y + y\Delta_x}{y(y - \Delta_y)} \quad \text{és} \quad \delta_{x/y} := \frac{\delta_x + \delta_y}{1 - \delta_y}$$

számok az osztás hiba- ill. relatív hibakorlátai.

## Bizonyítás

Elemi átalakításokat használva kapjuk

$$\left| \frac{x}{y} - \frac{\tilde{x}}{\tilde{y}} \right| = \frac{|x\tilde{y} - xy + xy - \tilde{x}y|}{y|\tilde{y}|} \leq \frac{x\Delta_y + y\Delta_x}{y|\tilde{y}|} = \frac{x\Delta_y + y\Delta_x}{y|y - (y - \tilde{y})|}.$$

A  $\delta_y < 1$  feltételből következik, hogy  $|y - \tilde{y}| \leq \Delta_y < y$ , ezért az  $|y - (y - \tilde{y})| \geq y - |y - \tilde{y}| \geq y - \Delta_y > 0$  egyenlőtlenség felhasználásával következik a tétel első állítása.

A második állítás igazolásához tekintsük

$$\frac{\left| \frac{x}{y} - \frac{\tilde{x}}{\tilde{y}} \right|}{\frac{x}{y}} = \frac{|x(\tilde{y} - y) - y(\tilde{x} - x)|}{x|\tilde{y}|} = \frac{\left| \frac{\tilde{y}-y}{y} - \frac{\tilde{x}-x}{x} \right|}{\left| 1 - \frac{y-\tilde{y}}{y} \right|} \leq \frac{\delta_x + \delta_y}{1 - \delta_y}.$$

Ha  $\delta_y$  kicsi, akkor

$$\delta_{x/y} \approx \delta_x + \delta_y.$$

Hasonlóan, ha  $\Delta_y$   $y$ -hoz képest elhanyagolható, akkor

$$\Delta_{x/y} \approx \frac{1}{y} \Delta_x + \frac{x}{y^2} \Delta_y.$$

Ha  $y$  sokkal kisebb, mint  $x$ , illetve ha  $y$  közel van 0-hoz, akkor  $\Delta_y$  ill.  $\Delta_x$  együtthatója nagy, azaz a hiba a tényezők hibáinak többszöröse lehet.

## Példa

Legyen  $x = 42.721531$ ,  $\tilde{x} = 42.721534$ ,  $y = 0.00324721$  és  $\tilde{y} = 0.00324732$ . Ekkor

$$\Delta_x = 3 \cdot 10^{-6} \quad \text{és} \quad \Delta_y = 1.1 \cdot 10^{-7}.$$

Másrészt

$$\frac{x}{y} \approx 13156.38071, \quad \frac{\tilde{x}}{\tilde{y}} \approx 13155.93597,$$

és így

$$\Delta_{x/y} \approx 0.44474.$$

Ekkor azt látjuk, hogy az osztás eredményének a hibája sokkal nagyobb, mint az  $x$  és  $y$  hibája.

## 1.4. A véges számábrázolás következményei

## Példa

Oldjuk meg az

$$x^2 - 83.5x + 1.5 = 0$$

másodfokú egyenletet négyjegyű aritmetikát használva:

$$\tilde{x} = \frac{83.5 \pm \sqrt{83.5^2 - 4 \cdot 1.5}}{2} = \frac{83.5 \pm \sqrt{6972 - 6.000}}{2} = \frac{83.5 \pm 83.46}{2},$$

így

$$\tilde{x}_1 = \frac{167.0}{2} = 83.50, \quad \text{és} \quad \tilde{x}_2 = \frac{0.040}{2} = 0.020.$$

Az egyenlet pontos megoldása (néhány tizedesjegy pontossággal)

$$x_1 = 83.482032 \quad \text{ill.} \quad x_2 = 0.0179679,$$

ezért a két gyök közelítésének relatív hibái

$$\delta_1 = 0.0002152 \quad \text{és} \quad \delta_2 = 0.113096.$$

Tekintsük az  $ax^2 + bx + c = 0$  másodfokú egyenlet két gyöke közül az

$$x_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}. \quad (1)$$

gyököt. Amikor  $b$  negatív, és  $4ac$  sokkal kisebb, mint  $b^2$ , két egymáshoz közeli számot vonunk ki egymásból a számlálóban, azaz fellép az értékes számjegyek végzetes elvesztésének jelensége. Ennek kiküszöbölésére gyöktelenítsük a számlálót:

$$x_2 = \frac{b^2 - (b^2 - 4ac)}{2a(-b + \sqrt{b^2 - 4ac})} = \frac{2c}{-b + \sqrt{b^2 - 4ac}}. \quad (2)$$

Ez a képlet algebrailag ekvivalens az (1) formulával. A különbség viszont az, hogy ebben nem szerepel kivonás (a nevezőben két pozitív számot adunk össze). Ha  $b$  pozitív, akkor a másik gyökképlettel ismételhetjük meg ugyanezt a trükköt, és kaphatjuk az

$$x_1 = \frac{2c}{-b - \sqrt{b^2 - 4ac}} \quad (3)$$

formulát.

## Példa

Számítsuk ki az előző példa második gyökét újra, négyjegyű aritmetikát és a gyökképlet (3) alakját használva:

$$\tilde{x}_2 = \frac{2 \cdot 1.5}{83.5 + \sqrt{83.5^2 - 4 \cdot 1.5}} = \frac{3}{83.5 + 83.46} = \frac{3}{167.0} = 0.01796.$$

Ennek a numerikus gyöknek a relatív hibája  $\delta_2 = 0.00044$ .

## Példa

Tegyük fel, hogy a

$$\cos^2 x - \sin^2 x.$$

kifejezést kell kiértékelnünk. Ha  $x = \frac{\pi}{4}$ , akkor a kifejezés pontos értéke 0, azaz ha  $x$   $\frac{\pi}{4}$ -hez közel van, akkor a kifejezés két egymáshoz közeli szám különbsége lesz, ahol fellép a pontosság elvesztése. Ezt könnyen kikerülhetjük, ha az eredeti kifejezés helyett az azzal algebrailag ekvivalens

$$\cos^2 x - \sin^2 x = \cos 2x$$

alakot használjuk.

## Példa

Tekintsük az  $f(x) = e^x - 1$  függvényt. Az  $x = 0$  közelében ismét két közel azonos számot kell egymásból kivonni, viszont most nincs olyan azonosság, amellyel ezt el lehetne kerülni. Ha  $e^x$  Taylor-sorát vesszük, akkor az 1-gyel való kivonással tudunk egyszerűsíteni:

$$\begin{aligned} f(x) &= 1 + x + \frac{x}{2} + \frac{x^3}{3!} + \cdots + \frac{x^n}{n!} + \cdots - 1 \\ &= x + \frac{x}{2} + \frac{x^3}{3!} + \cdots + \frac{x^n}{n!} + \cdots . \end{aligned}$$

Tehát  $f$ -et érdemes ennek a végtelen sornak egy véges közelítő összege segítségével kiértékelni.

Egy más jellegű problémát vet fel a következő példa.

## Példa

Számítsuk ki az

$$y = \frac{15^{40}}{40!}$$

értékét. A probléma a következő: ha a képlet alapján először a számlálót és a nevezőt külön akarjuk kiszámolni, rögtön beleütközünk a számábrázolás szabta korlátokba, egyszeres pontosság használata esetén már túlcsordul a számolás. Másrészt tudjuk, hogy  $a^n/n! \rightarrow 0$ , ha  $n \rightarrow \infty$ , így a számolás végeredménye várhatóan kis szám lesz. Rendezzük úgy a számítást, hogy minden részeredmény benne maradjon az ábrázolható számok tartományában:

$$\frac{15^{40}}{40!} = \frac{15}{40} \cdot \frac{15}{39} \cdot \frac{15}{38} \cdots \frac{15}{1}.$$

## Példa folyt.

Ezt a képletet a számítógépen egy egyszerű **for** ciklussal kiszámolhatjuk:

$y \leftarrow 15$

**for**  $i = 2, \dots, 40$  **do**

$$y \leftarrow y \cdot \frac{15}{i}$$

**end do**

**output**( $y$ )

Az eredmény 0.135521.

## Példa

Számítsuk ki az

$$A = 1.000 + 0.0003 + 0.0003 + \cdots + 0.0003 = 1.000 + \sum_{i=1}^{1000} 0.0003$$

összeget, **négyjegyű aritmetikát** használva! Balról jobbra értékeljük ki az összeadásokat, így először az  $1.000 + 0.0003$  összeget kell kiszámítanunk. Négyjegyű aritmetika szerint

$$1.000 + 0.0003 = 1.0003 = 1.000$$

kerekítés után. Ehhez hozzáadva a következő számot, a 4 jegyre kerekítés miatt, újra  $1.000 + 0.0003 + 0.003 = 1.000$  lesz. Látható, hogy  $A = 1.000$  lesz a számolás eredménye.

## Példa folyt.

Nézzük most újra az előbbi összeadást, de más sorrendben:

$$B = 0.0003 + 0.0003 + \cdots + 0.0003 + 1.000 = \sum_{i=1}^{1000} 0.0003 + 1.000.$$

Most először számítsuk ki

$$0.0003 + 0.0003 = 0.0006.$$

Ezután:

$$0.0003 + 0.0003 + 0.0003 = 0.0009,$$

és hasonlóan

$$B = \sum_{i=1}^{1000} 0.0003 + 1.000 = 0.3 + 1.000 = 1.300.$$

A kettőnél több tagú összeadás nem kommutatív a számítógépeken!

Amikor lehet, az összeadásokat érdemes a tagok növekvő sorrendjében végezni.